

Automatisierter Testbericht

Inventory Management Projekt

Test-Framework: pytest

Verwendeter Befehl: `python -m pytest app/tests -v`

Testergebnis

- 28 Tests gesammelt
- 28 Tests erfolgreich bestanden
- 0 fehlgeschlagene Tests
- 0 Fehler

Zusammenfassung: Die automatisierten Tests überprüfen die zentrale Service-Logik des aktualisierten Inventory-Management-Systems.

Die Tests wurden an die neueste main-Branch-Struktur angepasst und decken folgende Services ab:

- CategoryServices
- LocationServices
- ProductServices
- UserService
- MovementService

Testkategorien

- Tests für den Category-Service
- Tests für den Location-Service
- Tests für den Product-Service
- Tests für den User-Service
- Tests für den Movement-Service

Verwendete Testdaten: Für jeden Test wird mithilfe von pytest Fixtures eine saubere In-Memory-SQLite-Datenbank erstellt.

Die Basis-Testdaten enthalten:

- eine Kategorie
- einen Lagerort
- einen Admin-Benutzer
- ein Produkt

Endergebnis: Alle 28 automatisierten Tests wurden erfolgreich bestanden.

Übersicht der automatisierten Testfälle

Testfall-ID	Beschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC_001	Kategorie erfolgreich erstellen	Kategorie wird erfolgreich erstellt	Erfolgreich	Bestanden
TC_002	Kategorie ohne Namen erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_003	Kategorie erfolgreich abrufen	Kategorie wird korrekt zurückgegeben	Erfolgreich	Bestanden
TC_004	Kategorie erfolgreich aktualisieren	Kategorie wird korrekt aktualisiert	Erfolgreich	Bestanden
TC_005	Kategorie erfolgreich löschen	Kategorie wird erfolgreich gelöscht	Erfolgreich	Bestanden
TC_006	Lagerort erfolgreich erstellen	Lagerort wird erfolgreich erstellt	Erfolgreich	Bestanden
TC_007	Lagerort ohne Namen erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_008	Lagerort erfolgreich abrufen	Lagerort wird korrekt zurückgegeben	Erfolgreich	Bestanden
TC_009	Lagerort erfolgreich aktualisieren	Lagerort wird korrekt aktualisiert	Erfolgreich	Bestanden
TC_010	Lagerort erfolgreich löschen	Lagerort wird erfolgreich gelöscht	Erfolgreich	Bestanden
TC_011	Bewegung erfolgreich erstellen	Bewegung wird erfolgreich erstellt	Erfolgreich	Bestanden
TC_012	Bewegung mit ungültigem Produkt erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_013	Bewegung mit ungültigem Benutzer erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_014	Bewegung mit ungültigem Lagerort erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_015	Bewegung erfolgreich abrufen	Bewegung wird korrekt zurückgegeben	Erfolgreich	Bestanden
TC_016	Bewegung erfolgreich löschen	Bewegung wird erfolgreich gelöscht	Erfolgreich	Bestanden
TC_017	Produktbewegung zum gleichen Lagerort erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_018	Produktbewegung mit unzureichendem Bestand erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_019	Produkt erfolgreich erstellen	Produkt wird erfolgreich erstellt	Erfolgreich	Bestanden
TC_020	Produkterstellung mit ungültiger Kategorie erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden

TC_021	Produktverfügbarkeit prüfen	Produktverfügbarkeit gibt true zurück	Erfolgreich	Bestanden
TC_022	Produkt erfolgreich löschen	Produkt wird erfolgreich gelöscht	Erfolgreich	Bestanden
TC_023	Benutzer erfolgreich abrufen	Benutzer wird korrekt zurückgegeben	Erfolgreich	Bestanden
TC_024	Benutzer nicht gefunden erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_025	Login erfolgreich	Benutzeranmeldung funktioniert korrekt	Erfolgreich	Bestanden
TC_026	Login mit falschem Passwort erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden
TC_027	Benutzerrolle erfolgreich ändern	Benutzerrolle wird korrekt aktualisiert	Erfolgreich	Bestanden
TC_028	Rollenänderung mit ungültiger Rolle erzeugt Fehler	ValueError wird ausgelöst	Erfolgreich	Bestanden

Testfälle – Inventory Management Projekt

TC_001 – Kategorie erfolgreich erstellen

Feld	Details
Testfall-ID	TC_001
Titel/Beschreibung des Testfalls	Überprüfen, ob eine neue Kategorie mit gültigen Eingabedaten erfolgreich erstellt werden kann.
Voraussetzungen	Eine saubere Testdatenbank ist verfügbar.
Testschritte	1. CategoryServices erstellen. 2. create_category() mit gültigen Kategoriedaten aufrufen. 3. Kategorie in der Datenbank speichern. 4. Die erstellten Kategorienfelder überprüfen.
Testdaten/Eingaben	Kategorienname: Electronics, Typ: sale
Erwartetes Ergebnis	Die Kategorie wird erfolgreich erstellt und in der Datenbank gespeichert.
Tatsächliches Ergebnis	Die Kategorie wurde erfolgreich erstellt.
Status	Bestanden
Kommentare	Keine Probleme festgestellt.

TC_006 – Lagerort erfolgreich erstellen

Feld	Details
Testfall-ID	TC_006
Titel/Beschreibung des Testfalls	Überprüfen, ob ein neuer Lagerort mit gültigen Eingabedaten erfolgreich erstellt werden kann.
Voraussetzungen	Eine saubere Testdatenbank ist verfügbar.
Testschritte	1. LocationServices erstellen. 2. create_location() mit gültigen Lagerortdaten aufrufen. 3. Lagerort in der Datenbank speichern. 4. Die erstellten Lagerortfelder überprüfen.
Testdaten/Eingaben	Lagerortname: Main Warehouse, Regalnummer: A1
Erwartetes Ergebnis	Der Lagerort wird erfolgreich erstellt und in der Datenbank gespeichert.
Tatsächliches Ergebnis	Der Lagerort wurde erfolgreich erstellt.

Status	Bestanden
Kommentare	Keine Probleme festgestellt.

TC_011 – Bewegung erfolgreich erstellen

Feld	Details
Testfall-ID	TC_011
Titel/Beschreibung des Testfalls	Überprüfen, ob eine Lagerbewegung mit gültigen Produkt-, Benutzer- und Lagerortdaten erfolgreich erstellt werden kann.
Voraussetzungen	Eine saubere Testdatenbank ist verfügbar. Ein Produkt, ein Benutzer und ein Lagerort existieren bereits.
Testschritte	1. MovementService erstellen. 2. Vorhandenes Produkt, Benutzer und Lagerort aus den Basis-Testdaten auswählen. 3. create_movement() mit gültigen Bewegungsdaten aufrufen. 4. Die erstellten Bewegungsfelder überprüfen.
Testdaten/Eingaben	Produkt: Laptop, Benutzer: admin, Lagerort: Main Warehouse, Menge: 3, Bewegungstyp: move
Erwartetes Ergebnis	Ein neuer Lagerbewegungseintrag wird erfolgreich in der Datenbank erstellt.
Tatsächliches Ergebnis	Die Lagerbewegung wurde erfolgreich erstellt.
Status	Bestanden
Kommentare	Keine Probleme festgestellt.

TC_017 – Produktbewegung zum gleichen Lagerort erzeugt Fehler

Feld	Details
Testfall-ID	TC_017
Titel/Beschreibung des Testfalls	Überprüfen, ob das System verhindert, dass ein Produkt zum gleichen Lagerort bewegt wird.
Voraussetzungen	Ein Produkt und ein Lagerort existieren bereits in der Datenbank.
Testschritte	1. MovementService erstellen. 2. Vorhandenes Produkt und Lagerort auswählen. 3. move_product() mit gleichem Quell- und Ziellagerort aufrufen. 4. Prüfen, ob ein Fehler ausgelöst wird.
Testdaten/Eingaben	Produkt: Laptop, Quelllagerort: Main Warehouse, Ziellagerort: Main Warehouse, Menge: 1
Erwartetes Ergebnis	Das System löst einen ValueError mit der Meldung „Source and target locations must be different“ aus.
Tatsächliches Ergebnis	ValueError wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Die Validierung der Geschäftsregel funktioniert korrekt.

TC_019 – Produkt erfolgreich erstellen

Feld	Details
Testfall-ID	TC_019
Titel/Beschreibung des Testfalls	Überprüfen, ob ein neues Produkt mit gültigen Eingabedaten erfolgreich erstellt werden kann.
Voraussetzungen	Eine gültige Kategorie und ein gültiger Lagerort existieren bereits in der Testdatenbank.
Testschritte	1. ProductServices erstellen. 2. create_product() mit gültigen Produktdaten aufrufen. 3. Produkt in der Datenbank speichern. 4. Die erstellten Produktfelder überprüfen.

Testdaten/Eingaben	Produktname: Keyboard, Menge: 5, Mindestbestand: 1, Kategorie: Electronics
Erwartetes Ergebnis	Das Produkt wird erfolgreich erstellt und in der Datenbank gespeichert.
Tatsächliches Ergebnis	Das Produkt wurde erfolgreich erstellt.
Status	Bestanden
Kommentare	Keine Probleme festgestellt.

TC 020 – Produkterstellung mit ungültiger Kategorie erzeugt Fehler

Feld	Details
Testfall-ID	TC_020
Titel/Beschreibung des Testfalls	Überprüfen, ob das System die Produkterstellung mit einer ungültigen Kategorie-ID verhindert.
Voraussetzungen	ProductServices ist verfügbar.
Testschritte	1. ProductServices erstellen. 2. create_product() mit ungültiger Kategorie-ID aufrufen. 3. Prüfen, ob ein Fehler ausgelöst wird.
Testdaten/Eingaben	Ungültige category_id: 999
Erwartetes Ergebnis	Das System löst einen ValueError mit der Meldung „Category not found“ aus.
Tatsächliches Ergebnis	ValueError wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Die Validierung funktioniert korrekt.

TC_025 – Login erfolgreich

Feld	Details
Testfall-ID	TC_025
Titel/Beschreibung des Testfalls	Überprüfen, ob sich ein Benutzer mit korrekten Zugangsdaten erfolgreich anmelden kann.
Voraussetzungen	Ein gültiger Benutzer mit gehashtem Passwort existiert in der Datenbank.
Testschritte	1. UserService erstellen. 2. login() mit gültigem Benutzernamen und Passwort aufrufen. 3. Das zurückgegebene Benutzerobjekt überprüfen.
Testdaten/Eingaben	Benutzername: admin, Passwort: password123
Erwartetes Ergebnis	Der Benutzer wird erfolgreich authentifiziert.
Tatsächliches Ergebnis	Die Anmeldung wurde erfolgreich durchgeführt.
Status	Bestanden
Kommentare	Die Authentifizierung funktioniert korrekt.

TC_026 – Login mit falschem Passwort erzeugt Fehler

Feld	Details
Testfall-ID	TC_026
Titel/Beschreibung des Testfalls	Überprüfen, ob das System Anmeldeversuche mit falschem Passwort ablehnt.
Voraussetzungen	Ein gültiger Benutzer existiert in der Datenbank.
Testschritte	1. UserService erstellen. 2. login() mit falschem Passwort aufrufen. 3. Prüfen, ob ein Fehler ausgelöst wird.
Testdaten/Eingaben	Benutzername: admin, Passwort: wrongpassword
Erwartetes Ergebnis	Das System löst einen ValueError mit der Meldung „Incorrect password“ aus.

Tatsächliches Ergebnis	ValueError wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Die Passwortvalidierung funktioniert korrekt.

Testergebnis der Testausführung

```

PS C:\Users\aadat\Desktop\inventory-management-fhnw> python -m pytest app/tests -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\aadat\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\aadat\Desktop\inventory-management-fhnw
configfile: pytest.ini
plugins: anyio-4.12.1, cov-7.1.0
collected 28 items

app/tests/test_category_services.py::test_create_category_success PASSED [ 3%]
app/tests/test_category_services.py::test_create_category_without_name_raises_error PASSED [ 7%]
app/tests/test_category_services.py::test_get_category_success PASSED [ 10%]
app/tests/test_category_services.py::test_update_category_success PASSED [ 14%]
app/tests/test_category_services.py::test_delete_category_success PASSED [ 17%]
app/tests/test_location_services.py::test_create_location_success PASSED [ 21%]
app/tests/test_location_services.py::test_create_location_without_name_raises_error PASSED [ 25%]
app/tests/test_location_services.py::test_get_location_success PASSED [ 28%]
app/tests/test_location_services.py::test_update_location_success PASSED [ 32%]
app/tests/test_location_services.py::test_delete_location_success PASSED [ 35%]
app/tests/test_movement_services.py::test_create_movement_success PASSED [ 39%]
app/tests/test_movement_services.py::test_create_movement_with_invalid_product_raises_error PASSED [ 42%]
app/tests/test_movement_services.py::test_create_movement_with_invalid_user_raises_error PASSED [ 46%]
app/tests/test_movement_services.py::test_create_movement_with_invalid_location_raises_error PASSED [ 50%]
app/tests/test_movement_services.py::test_get_movement_success PASSED [ 53%]
app/tests/test_movement_services.py::test_delete_movement_success PASSED [ 57%]
app/tests/test_movement_services.py::test_move_product_same_location_raises_error PASSED [ 60%]
app/tests/test_movement_services.py::test_move_product_not_enough_stock_raises_error PASSED [ 64%]
app/tests/test_product_services.py::test_create_product_success PASSED [ 67%]
app/tests/test_product_services.py::test_create_product_with_invalid_category_raises_error PASSED [ 71%]
app/tests/test_product_services.py::test_check_availability_returns_true PASSED [ 75%]
app/tests/test_product_services.py::test_delete_product_success PASSED [ 78%]
app/tests/test_user_service.py::test_get_user_success PASSED [ 82%]
app/tests/test_user_service.py::test_get_user_not_found_raises_error PASSED [ 85%]
app/tests/test_user_service.py::test_login_success PASSED [ 89%]
app/tests/test_user_service.py::test_login_wrong_password_raises_error PASSED [ 92%]
app/tests/test_user_service.py::test_change_role_success PASSED [ 96%]
app/tests/test_user_service.py::test_change_role_invalid_role_raises_error PASSED [100%]

===== 28 passed in 0.67s =====
PS C:\Users\aadat\Desktop\inventory-management-fhnw>

```

Alle automatisierten Tests wurden erfolgreich mit pytest ausgeführt.